

- **Base de Données NoSQL : MongoDB** -

TP 2 : Principe de ReplicaSet

Le **but** de ce TP est de tester le **système de tolérance aux pannes de MongoDB**. Pour cela, nous allons lancer des serveurs avec des ReplicaSets.

- Un ReplicaSet est un groupe d'instances qui maintiennent le même ensemble de données. Il se compose d'un nœud primaire et de nœuds secondaires. Les opérations ont lieu sur le nœud primaire, puis elles sont reproduites sur les nœuds secondaires.
- Dans le cas où le nœud primaire ne répond pas pendant 10 secondes, un nœud secondaire est élu comme nouveau nœud primaire.
- Pour s'assurer qu'une élection désigne toujours un maître, MongoDB permet de lancer un serveur *mongod* en mode « arbitre » : il ne stocke pas de données et consomme très peu de RAM, il sert juste à atteindre le nombre de votants nécessaires pour l'élection.
- Un *ReplicaSet* doit contenir au minimum 3 serveurs (1 Primary et 2 Secondary) pour garantir un minimum de tolérance aux pannes. Il peut contenir jusqu'à 50 serveurs.

Nous allons créer ici un replicaSet composé de 4 serveurs, dont 1 primaire, 2 secondaires et un arbitre.

Partie I : Configuration d'un ReplicaSet

1. Pour chaque serveur, créer un répertoire de données:
/data/r0s1, /data/r0s2, /data/r0s3 et /data/arb
2. Choisir un nom de réplication : **rs0**
3. Ouvrir 3 nouvelles consoles. Dans chaque console, lancer un replicat avec un nouveau port de connexion :
 - **mongod --replSet rs0 --dbpath /data/r0s1 --port 27018**
 - **mongod --replSet rs0 --dbpath /data/r0s2 --port 27019**
 - **mongod --replSet rs0 --dbpath /data/r0s3 --port 27020**

4. Ouvrir une nouvelle console pour se connecter au serveur qui sera choisi comme serveur primaire: **mongo --port 27018**
5. Lancer le mode de réplcation : **rs.initiate()**;
(Dans la console “mongo” apparaîtra *rs0:PRIMARY*> désignant le serveur primaire)
6. Rajouter chaque réplicat à votre ReplicaSet: **rs.add("localhost:port");**
7. Vérifier la configuration de l’ensemble des serveurs : **rs.conf()**;
8. Lorsqu’il y a plusieurs réplicats, il est nécessaire de définir un arbitre dans le cas où le serveur primaire tombe en panne. Ouvrir une nouvelle console :
mongod --replSet rs0 --dbpath /data/arb --port 30000
9. Ajouter l’arbitre dans la console mongo *rs0:PRIMARY*>
rs.addArb(“localhost:30000”)
10. Pour consulter le statut actuel du ReplicaSet, il suffit d’utiliser l’instruction suivante: **rs.status()** ;

Partie II : Principe de réplcation de données sur un ReplicaSet

Après avoir configuré le ReplicaSet *rs0*, il faut tester la duplication de données sur les différents réplicats via les étapes suivantes :

1. Importer le dataset du 1^{er} TP (*books.json*) dans le serveur primaire.
2. Vérifier l’existence des données sur le serveur primaire (*show dbs ...*)
3. Se connecter à un serveur secondaire du réplicatset *rs0* et consulter les données importées.
6. Dans la console *rs0:PRIMARY*>, ajouter un document “{ test : 1 }”.
7. Dans la console *rs0:SECONDARY*>, vérifier son existence avec une requête **find**.

Partie III : La Tolérance aux Pannes

1. Dans la console *rs0:PRIMARY*>, ajouter un document “{test : 2}” à la collection *books*.
2. Pour simuler une panne de serveur primaire, fermez le processus correspondant. Dans ce cas, l’arbitre désignera automatiquement le serveur qui deviendra le primaire (*cette opération peut prendre un peu de temps* sinon forcer cette opération avec **rs.stepDown()**).
3. Consulter la nouvelle configuration des serveurs par la commande: **rs.status()**;

4. Lancer le nouveau serveur primaire.
5. Vérifier l'existence de votre document dans la collection **books**.

Partie IV : Manipulation graphique d'un Réplicaset

1. Se connecter à votre BD MongoDB graphique et créer une nouvelle connexion en mode répliquet.
2. Ajouter les serveurs configurés à cette nouvelle connexion.
3. Vérifier votre configuration.